Freight Rate Prediction

Validation approach, modeling report, and December 2025 forecast

1. Data Overview

Labeled development data (`train-test.csv`) contains 48,000 loads dated January 1 - October 31, 2025, with 14 columns: route (pickup/delivery city and lat/lon), distance, equipment type, weight, date, two market signals (`market_index` , `quote_signal`), and the target `posted_rate` . No duplicate rows or duplicate `load_id` values were found.

The two prediction targets both fall **outside** this date range: the 12,000-load validation set spans November 1 - December 31, 2025, and the fixed-route December chart inputs cover December 1-31, 2025 - a genuine extrapolation problem, not interpolation, which shaped both the train/validation split and the final model choice (see Sections 3 and 4).

2. Data Quality Issues Identified

- **Negative weight values (sign error).** 292 of 48,000 rows had negative `weight` (and 145 of 12,000 in validation.csv). Weight is physically non-negative, and the magnitude distribution of the negative values matched the positive distribution almost exactly (e.g. mean magnitude ~31,724 lb vs. mean positive weight ~31,415 lb) - consistent with a sign-flip bug rather than corrupted or missing data. Fixed with `abs()` , applied identically to the training data, validation.csv, and the December inputs.
- **Missing values.** `weight` was missing in 300 training rows (165 in validation); `market_index` was missing in 374 training rows (249 in validation). Both are imputed with the training-set median inside the model pipeline, so validation and December data are filled using statistics learned only from training data - never their own.
- **Unseen categories in validation.** 8 pickup/delivery cities in validation.csv never appear in train-test.csv (Laredo, Allentown, Chicago, Knoxville, Charlotte, Norfolk, San Diego, Jackson). The encoder (`OneHotEncoder(handle_unknown="ignore")`) treats these as all-zero rather than erroring or crashing at inference time.
- `market_index` / `quote_signal` **are absent from the December chart inputs file entirely** (it only has pickup, delivery, distance,

equipment, weight, date). Rather than building a separate fallback model just for December, the final model drops both features everywhere, keeping one consistent feature set across validation.csv and the December inputs (see Section 4).

3. Train / Validation Split Approach

Split is **time-based**, not random: training rows dated before October 1, 2025 (43,147 rows) vs. a holdout of the remaining rows, October 1-31, 2025 (4,853 rows).

Reasoning: a random split would let the model see examples from every month, including ones adjacent to any given test row, which overstates how well it will generalize to dates it has genuinely never seen. Since the real prediction targets (validation.csv, December) fall entirely after the training data's date range, a time-based holdout - train on the past, validate on the most recent slice - is a much closer proxy for the actual extrapolation the final model has to do.

Exploratory analysis supported this choice: monthly average `posted_rate` drifts upward across the year (January ~$2,256 to June ~$2,497), confirming the problem has a real time/seasonal component that a naive split would hide.

4. Feature Engineering & Model Selection

All models share the same preprocessing: median imputation for numeric features, most-frequent imputation + one-hot encoding (unknown categories ignored) for categorical features, inside a single `scikit-learn` `Pipeline` / `ColumnTransformer` so the exact same transform is applied to training, validation, and December data.

4.1 Model comparison (October holdout)

Model	Features	MAE	RMSE
Baseline linear regression	distance only	192.77	668.21
Linear regression	distance, weight, market_index, quote_signal, time_index, equipment, pickup, delivery	138.35	651.57
Random forest	same as above	233.42	748.04
Gradient boosting	same as above	142.63	653.39

Linear regression, engineered features	+ log(distance), distance×market_index, distance×quote_signal	153.12	652.82
Linear regression (final, submitted)	**distance, weight, time_index, equipment, pickup, delivery**	**172.67**	**655.74**

The 138.35 MAE row uses `market_index` / `quote_signal`, which don't exist in the December inputs - it is not the model being submitted. The final row is the actual holdout score of the model used to generate both submission files.

4.2 Why linear regression over tree-based models

Random forest and gradient boosting were tested with the same feature set and did not outperform linear regression on this holdout - but the more important reason for choosing linear regression is **extrapolation**. Trees split on the range of values they saw during training; `time_index` never exceeded ~303 (October 31) during training, so a tree-based model has no way to represent dates further in the future than that and would flatten out instead of continuing the trend into November/December. Linear regression has no such ceiling - it carries the learned per-day trend forward linearly, which matches the real task (forecasting 31-91 days past the last training date) far more closely.

Feature engineering (log-distance, distance×market_index, distance×quote_signal interaction terms) was tested and did not improve on the plain full-feature linear model, so it was not carried into the final version.

4.3 Error analysis

On the October holdout, absolute error is small for most loads but has a long right tail: median $57.96, 90th percentile $189.92, 99th percentile $1,809.51. High-value, long-haul loads are the hardest to pin down precisely - consistent with the wide spread already visible in `posted_rate` for the longest routes.

5. Final Model & Submission

The final model is retrained on the full labeled dataset (48,000 rows, weight sign-fixed) using `distance`, `weight`, `time_index` (numeric) and `equipment`, `pickup`, `delivery` (categorical, one-hot) inside the same pipeline described above.

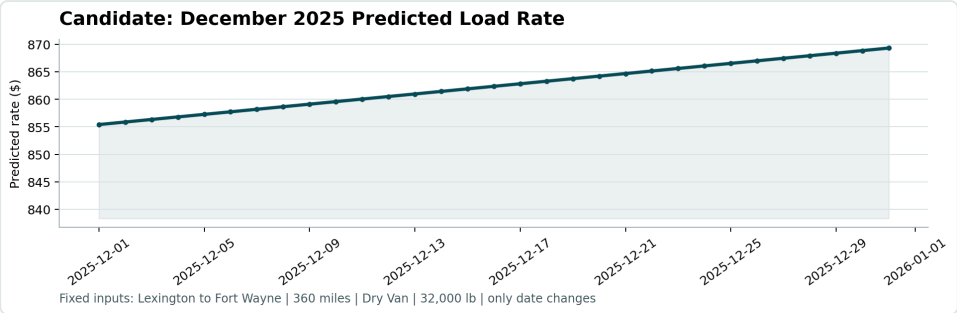
- `validation_predictions.csv` - 12,000 rows, predicted_rate range $185.85 - $6,630.37, no missing or non-positive values.

- `december_predictions.csv` - 31 rows (Dec 1-31, 2025), predicted_rate range $855.41 - $869.34.

Both files pass `score.py`'s structural validation.

6. December 2025 Forecast

Fixed inputs: Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb - only the date changes across the 31 rows. The chart below is produced by the provided `score.py` from `december_predictions.csv`.



The forecast rises smoothly day-over-day rather than flattening out, reflecting the upward trend already visible in train-test.csv (January-October) carried forward by the linear time term - the behavior a tree-based model could not have produced this far past its training date range.